```
bool pop_task_from_local_queue(task_type& task)
{
    return local_work_queue && local_work_queue->try_pop(task);
}

bool pop_task_from_pool_queue(task_type& task)
{
    return pool_work_queue.try_pop(task);
}

bool pop_task_from_other_thread_queue(task_type& task)     <-- 4
{
    for(unsigned i=0;i<queues.size();++i)
    {
        unsigned const index=(my_index+i+1)%queues.size();     <-- 5
        if(queues[index]->try_steal(task))
        {
            return true;
        }
    }

    return false;
}
```

This code is very similar to listing 9.6. The first difference is that each thread has a `work_stealing_queue` rather than a plain `std::queue<>` ❷. When each thread is created, rather than allocating its own work queue, the pool constructor allocates one ❻, which is then stored in the list of work queues for this pool ❶. The index of the queue in the list is then passed in to the thread function and used to retrieve the pointer to the queue ❸. This means that the thread pool can access the queue when trying to steal a task for a thread that has no work to do. `run_pending_task()` will now try to take a task from its thread's own queue ❼, take a task from the pool queue ❽, or take a task from the queue of another thread ❾.

`pop_task_from_other_thread_queue()` ❹ iterates through the queues belonging to all the threads in the pool, trying to steal a task from each in turn. In order to avoid every thread trying to steal from the first thread in the list, each thread starts at the next thread in the list, by offsetting the index of the queue to check by its own index ❺.